

VPN Implementation Project

Inter/Intranet Firewalls/E-Comm. Sec

Home Network VPN Server Deployment

Student Name:	<i>Eli Ecton</i>
Course:	Inter/Intranet Firewalls/E-Comm. Sec
Date:	April 29, 2026

Introduction

This project demonstrates the successful implementation of a Virtual Private Network (VPN) server on a home network, enabling secure remote access to a home network. The solution use WireGuard VPN deployed on an Ubuntu Linux virtual machine, with firewall configuration and network security measures. This report documents the complete setup process, network architecture, security configurations, and successful testing of the VPN infrastructure. The major steps needed to implement the VPN include configuration of port forwarding, network address translation (NAT), encrypted tunnels using modern protocols, and verification of secure remote access from external networks.

1.1 Overview

This project involves deploying a production VPN server to enable secure remote access to a home internal network. The implementation demonstrates practical application of firewall configuration, VPN technologies, and network security principles.

1.2 Objectives

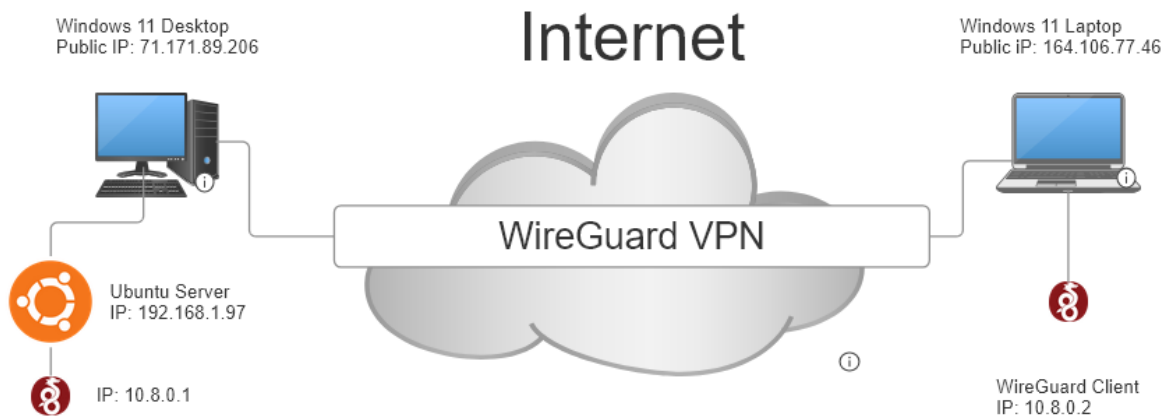
- Deploy a functional VPN server on home network infrastructure
- Configure firewall rules and port forwarding for secure external access
- Implement network address translation and IP forwarding
- Establish encrypted tunneling
- Verify remote connectivity
- Document the complete implementation process

2. Network Architecture

2.1 Network Topology - After VPN Implementation

The enhanced network architecture includes the VPN server infrastructure:

- Ubuntu Linux virtual machine running on desktop PC
- WireGuard VPN server software on Ubuntu VM
- Router configured with port forwarding for VPN traffic
- Virtual network interface (wg0) with dedicated subnet (10.8.0.0/24)
- Remote client capability for external network access



2.2 IP Addresses

Device/Network	IP Address	Description
Public IP	X.X.X.X	ISP-assigned external IP
Router	192.168.1.1	Gateway/Default route
Ubuntu VM (VPN Server)	192.168.1.97	VPN server on LAN
VPN Network	10.8.0.0/24	Virtual tunnel network
VPN Server Interface	10.8.0.1	WireGuard wg0 interface
VPN Client	10.8.0.2	Remote laptop when connected

3. Hardware and Software Infrastructure

3.1 Hardware Components

VPN Server Hardware:

- Desktop PC: Ryzen 7 9800x3d CPU, 32GB DDR5 RAM
- Hypervisor: VirtualBox
- VM Resources: 4096MB RAM, 2 Cores, 25GB Disk

Network Equipment:

- Router: Verizon Fios Router
- Network Connection: Ethernet

3.2 Software Components

Component	Software/Version	Purpose
Host OS	Windows 11	Desktop operating system
VM OS	Ubuntu Server 24.04 LTS	VPN server platform
VPN Software	WireGuard v1.0.2	VPN server application
Client Software	WireGuard for Windows	VPN client application

4. Services and Protocols

This section details the specific network services, protocols, and cryptographic methods employed in the VPN implementation. Understanding these technical specifications is critical for both security analysis and troubleshooting.

4.1 VPN Protocol and Transport

- **VPN Protocol:** WireGuard
- **Transport Protocol:** UDP (User Datagram Protocol)
- **Port Number:** 51820

4.2 Cryptographic Protocols

- **Encryption Algorithm:** ChaCha20Poly1305 (authenticated encryption with associated data)
- **Key Exchange:** Curve25519 (Elliptic-curve Diffie-Hellman)
- **Authentication Method:** Public/private key cryptography (asymmetric encryption)
- **Hashing:** BLAKE2s for cryptographic hashing
- **Key Size:** 256-bit keys for all cryptographic operations

4.3 Network Services

- **Virtual Network:** 10.8.0.0/24 (VPN tunnel subnet)
- **IP Forwarding:** Enabled via Linux kernel parameter (`net.ipv4.ip_forward=1`)
- **Network Address Translation (NAT):** iptables MASQUERADE for outbound traffic
- **DNS Resolution:** Google Public DNS (8.8.8.8, 8.8.4.4)
- **Port Forwarding:** Router-level forwarding of UDP 51820 to VPN server

4.4 Connection Persistence

- **PersistentKeepalive:** 25 seconds (maintains connection through NAT)

- **Purpose:** Prevents NAT timeout by sending periodic keepalive packets, ensuring continuous connectivity even during idle periods

4.5 Protocol Summary Table

Protocol/Service	Specification
VPN Protocol	WireGuard
Transport	UDP Port 51820
Encryption	ChaCha20Poly1305
Key Exchange	Curve25519 ECDH
Authentication	Public Key Cryptography
Hashing	BLAKE2s
VPN Network	10.8.0.0/24
DNS	Google Public DNS (8.8.8.8, 8.8.4.4)
NAT Method	iptables MASQUERADE
Keepalive	25 seconds

5. Implementation Process

5.1 Virtual Machine Setup

Step 1: VM Creation and Installation

- Downloaded Ubuntu Server 24.04 LTS ISO
- Created virtual machine with 4096 MB RAM, 2 cores, 25GB disk
- Configured network adapter in Bridged mode
- Installed Ubuntu Server with OpenSSH server enabled

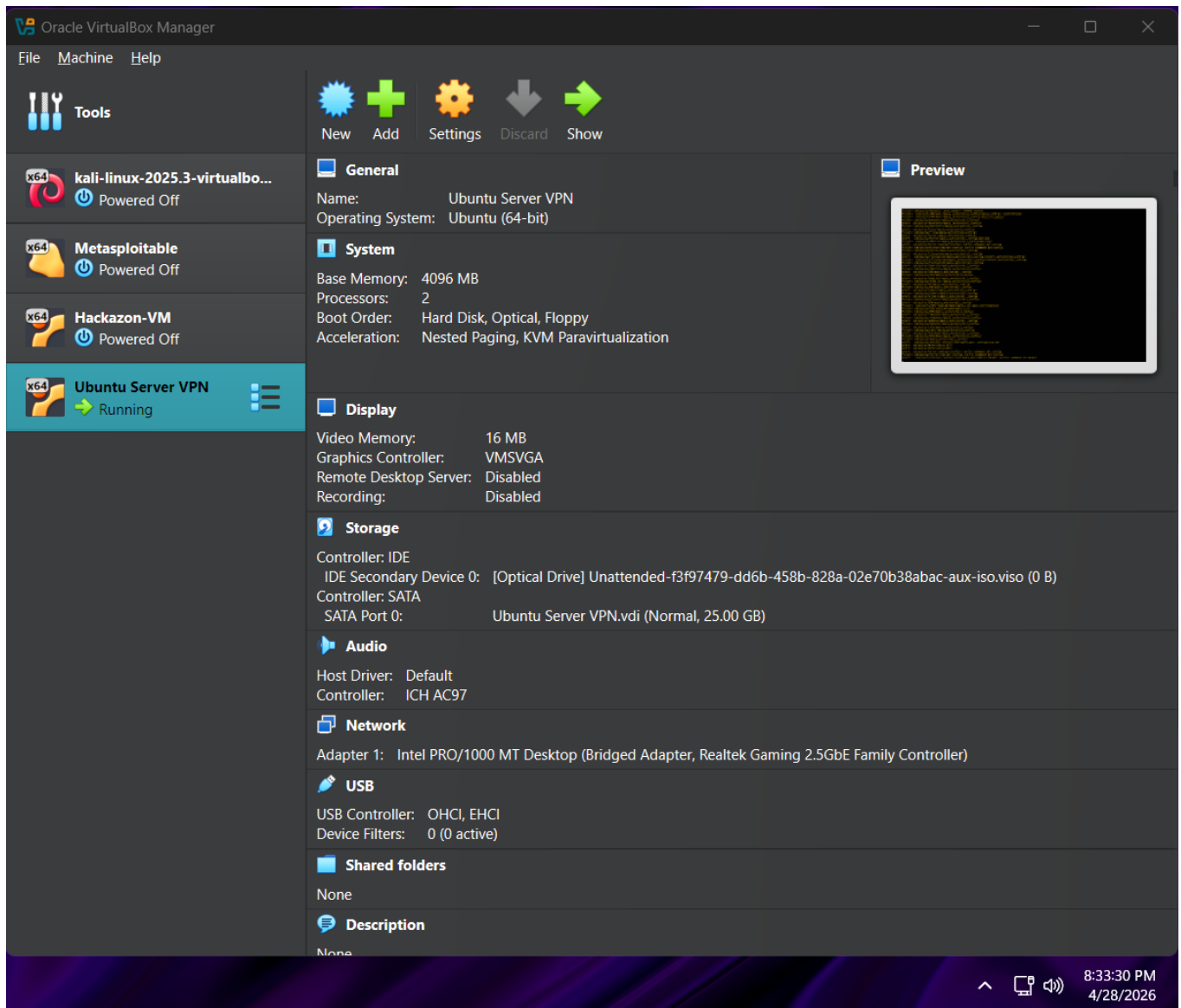


Figure 1: Ubuntu Server VM Configuration

Step 2: Network Configuration

- Identified network interface name using 'ip addr show' command
- Configured static IP address via netplan
- Set DNS servers to 8.8.8.8 and 8.8.4.4
- Verified network connectivity

```
vpnserver@UbuntuServerVPN:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
    link/ether 08:00:27:c8:e0:39 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.97/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 86332sec preferred_lft 86332sec
    inet6 fe80::a00:27ff:fec8:e039/64 scope link
        valid_lft forever preferred_lft forever
vpnserver@UbuntuServerVPN:~$
```

Figure 2: Results of ip addr show

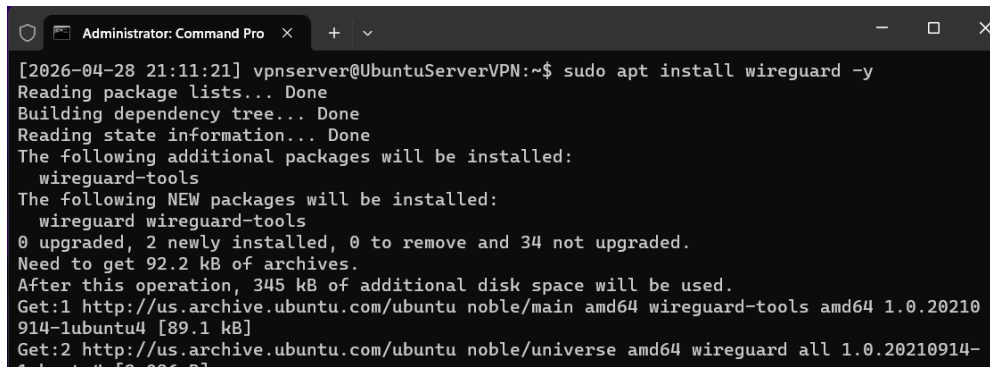
```
Administrator: Command Prom...
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.1.97/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify
9:09:42 PM
4/28/2026
```

Figure 3: Netplan Configuration

5.2 WireGuard Installation and Configuration

Step 1: Software Installation

- Updated package repositories: `sudo apt update`
- Installed WireGuard: `sudo apt install wireguard -y`

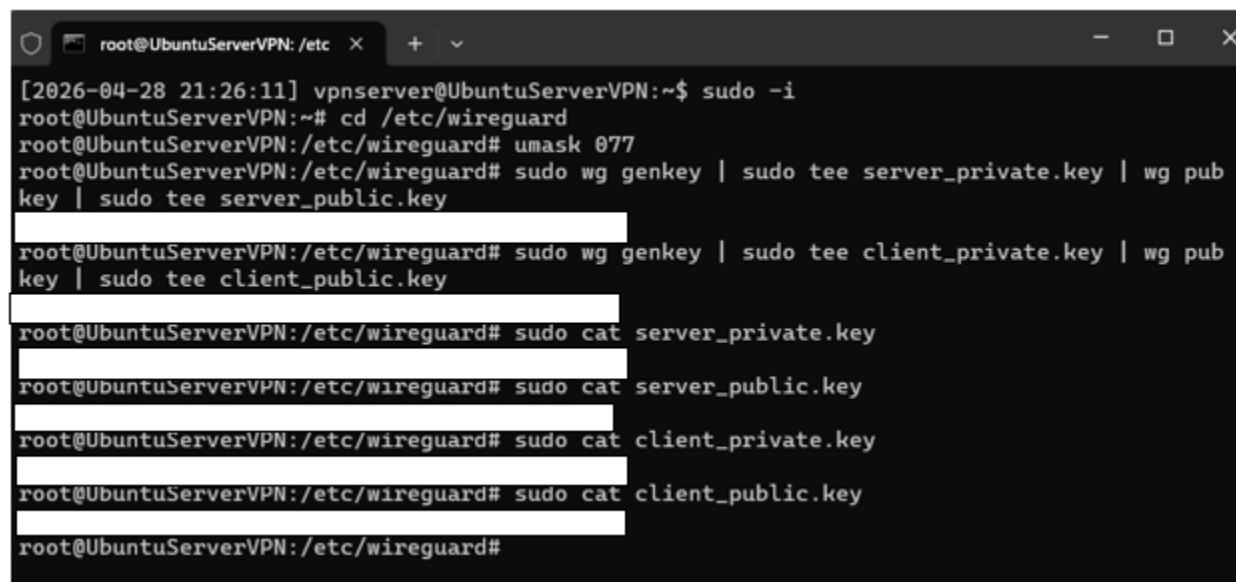


```
Administrator: Command Pro x + v
[2026-04-28 21:11:21] vpnserver@UbuntuServerVPN:~$ sudo apt install wireguard -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  wireguard-tools
The following NEW packages will be installed:
  wireguard wireguard-tools
0 upgraded, 2 newly installed, 0 to remove and 34 not upgraded.
Need to get 92.2 kB of archives.
After this operation, 345 kB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu noble/main amd64 wireguard-tools amd64 1.0.20210914-1ubuntu4 [89.1 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu noble/universe amd64 wireguard all 1.0.20210914-1ubuntu4 [3.1 kB]
debconf: delaying package configuration, since apt-utils is not installed
```

Figure 4: WireGuard Installation

Step 2: Cryptographic Key Generation

- Generated server private and public keys
- Generated client private and public keys
- Secured key files with appropriate permissions (umask 077)

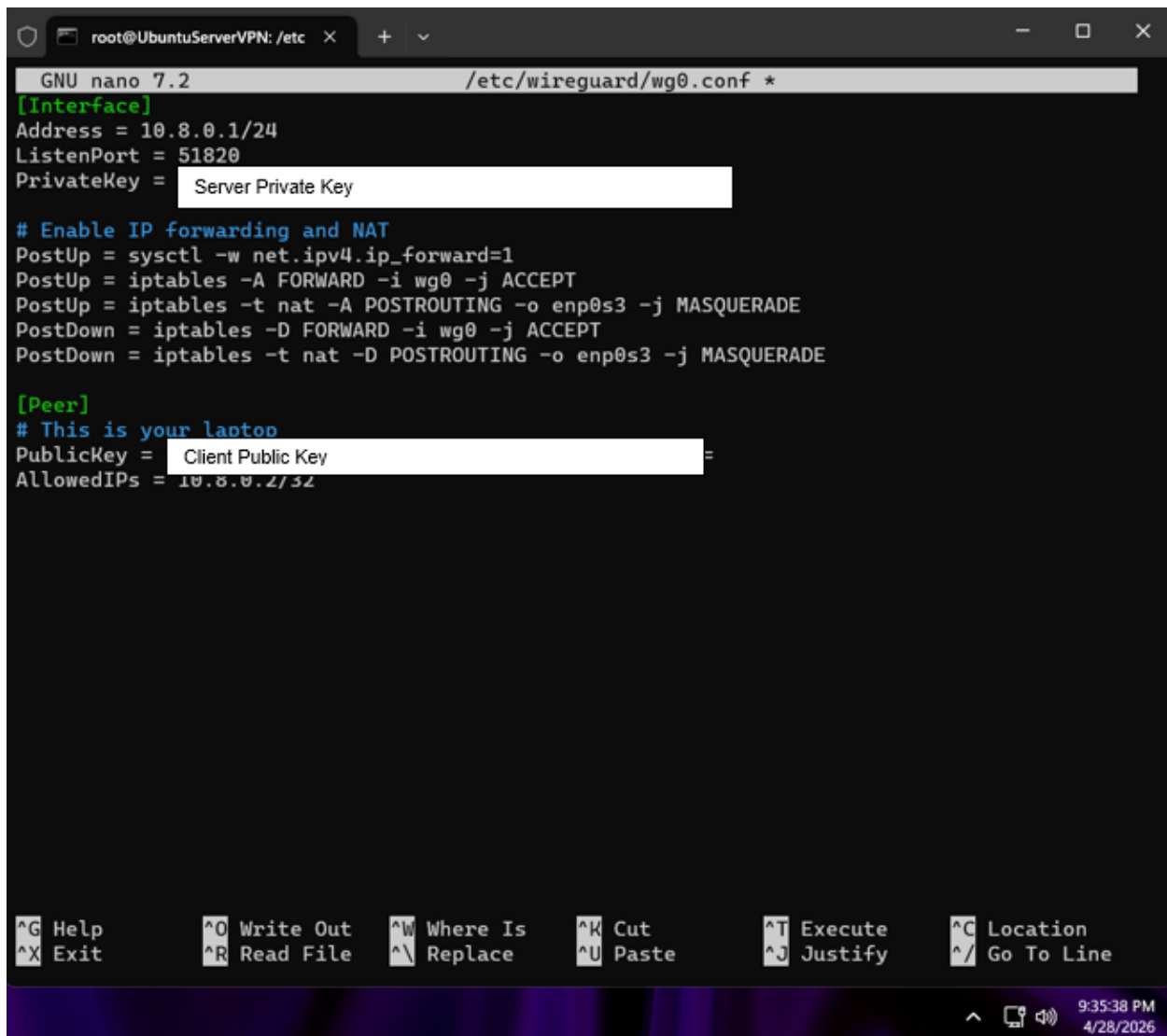


```
root@UbuntuServerVPN: /etc x + v
[2026-04-28 21:26:11] vpnserver@UbuntuServerVPN:~$ sudo -i
root@UbuntuServerVPN:~# cd /etc/wireguard
root@UbuntuServerVPN:/etc/wireguard# umask 077
root@UbuntuServerVPN:/etc/wireguard# sudo wg genkey | sudo tee server_private.key | wg pub
key | sudo tee server_public.key
[redacted]
root@UbuntuServerVPN:/etc/wireguard# sudo wg genkey | sudo tee client_private.key | wg pub
key | sudo tee client_public.key
[redacted]
root@UbuntuServerVPN:/etc/wireguard# sudo cat server_private.key
[redacted]
root@UbuntuServerVPN:/etc/wireguard# sudo cat server_public.key
[redacted]
root@UbuntuServerVPN:/etc/wireguard# sudo cat client_private.key
[redacted]
root@UbuntuServerVPN:/etc/wireguard# sudo cat client_public.key
[redacted]
root@UbuntuServerVPN:/etc/wireguard#
```

Figure 5: Umask and Key Generation

Step 3: Server Configuration File

Created /etc/wireguard/wg0.conf with the following configuration:

A screenshot of a terminal window titled 'root@UbuntuServerVPN: /etc'. The window shows the nano 7.2 editor editing the file '/etc/wireguard/wg0.conf'. The configuration is as follows:
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = [redacted]

Enable IP forwarding and NAT
PostUp = sysctl -w net.ipv4.ip_forward=1
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostUp = iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
PostDown = iptables -t nat -D POSTROUTING -o enp0s3 -j MASQUERADE

[Peer]
This is your laptop
PublicKey = [redacted]
AllowedIPs = 10.8.0.2/32
The terminal has a status bar at the bottom with various shortcuts like ^G Help, ^O Write Out, etc. The system clock in the bottom right corner shows 9:35:38 PM on 4/28/2026.

Figure 6: wg0.conf Configuration

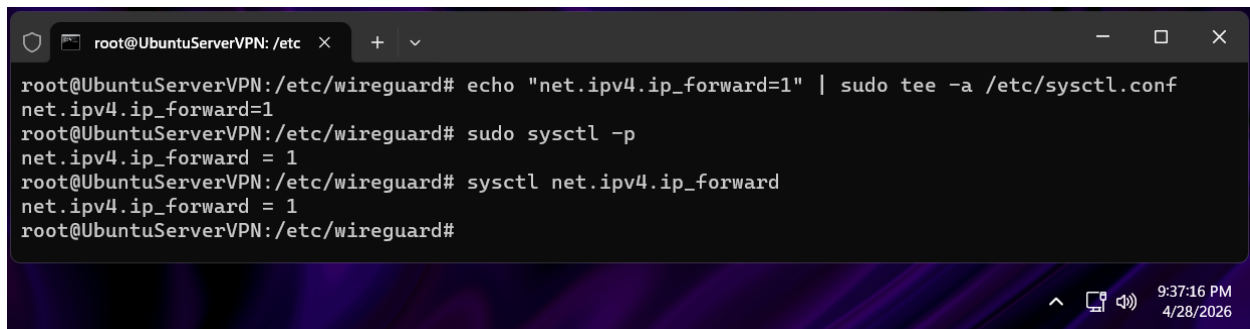
Configuration includes:

- Interface address: 10.8.0.1/24
- Listen port: 51820
- Server private key
- PostUp/PostDown rules for IP forwarding and NAT
- Client public key

5.3 Firewall and Network Configuration

Step 1: Enable IP Forwarding

- Enabled IPv4 forwarding
- Made persistent by adding to /etc/sysctl.conf

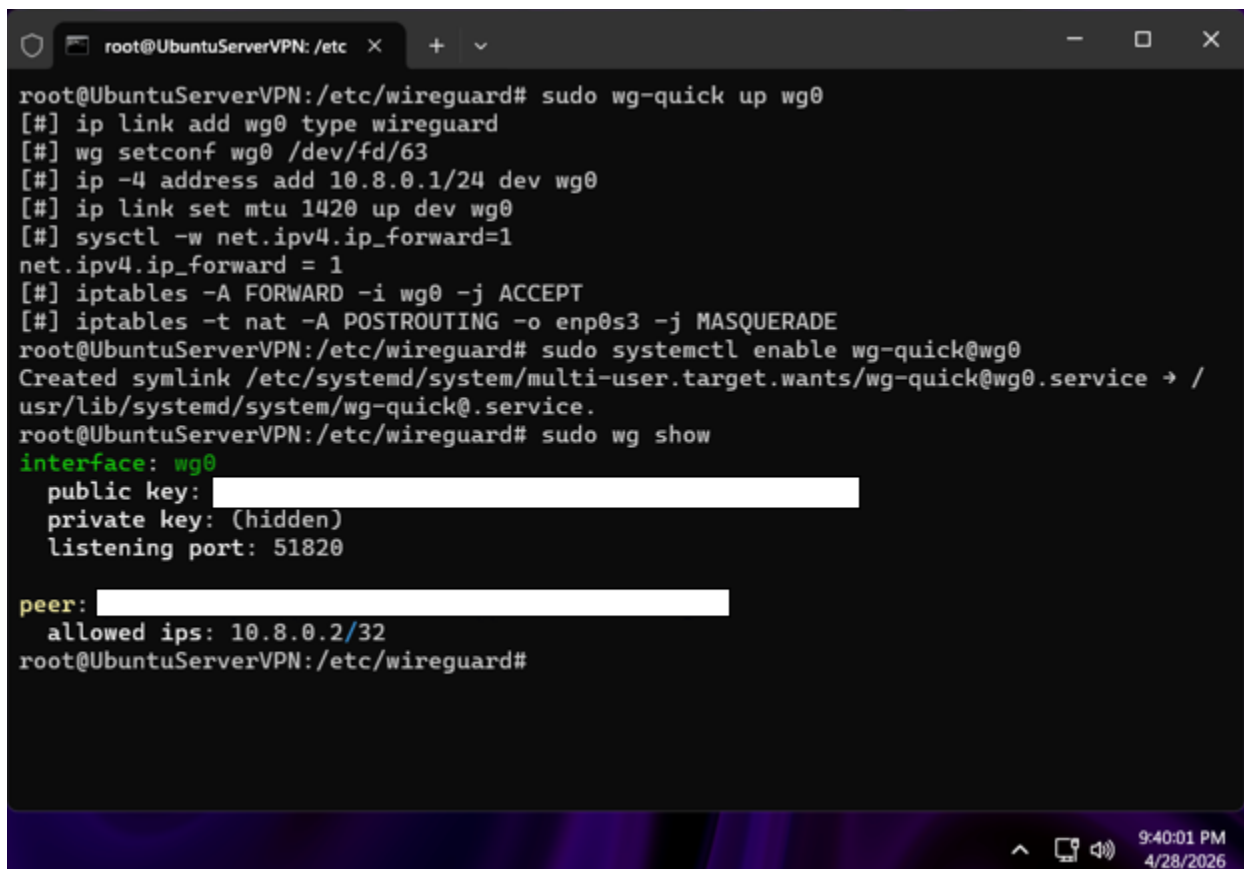
A terminal window titled 'root@UbuntuServerVPN: /etc' showing the configuration of IP forwarding. The user enters 'echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf', followed by 'sudo sysctl -p', and then 'sysctl net.ipv4.ip_forward'. The output shows 'net.ipv4.ip_forward = 1' being set.

```
root@UbuntuServerVPN:/etc/wireguard# echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
net.ipv4.ip_forward=1
root@UbuntuServerVPN:/etc/wireguard# sudo sysctl -p
net.ipv4.ip_forward = 1
root@UbuntuServerVPN:/etc/wireguard# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
root@UbuntuServerVPN:/etc/wireguard#
```

Figure 7: IP Forwarding

Step 2: WireGuard Service Activation

- Started WireGuard interface: `sudo wg-quick up wg0`
- Enabled automatic startup: `sudo systemctl enable wg-quick@wg0`
- Verified interface status: `sudo wg show`

A terminal window titled 'root@UbuntuServerVPN: /etc' showing the activation of the WireGuard interface. The user enters 'sudo wg-quick up wg0', which outputs several configuration steps including adding the interface, setting the IP address, and enabling IP forwarding. Then, the user enters 'sudo systemctl enable wg-quick@wg0', which outputs the creation of a symlink. Finally, the user enters 'sudo wg show', which displays the interface details and peer information.

```
root@UbuntuServerVPN:/etc/wireguard# sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.8.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
[#] iptables -A FORWARD -i wg0 -j ACCEPT
[#] iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
root@UbuntuServerVPN:/etc/wireguard# sudo systemctl enable wg-quick@wg0
Created symlink /etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service → /usr/lib/systemd/system/wg-quick@.service.
root@UbuntuServerVPN:/etc/wireguard# sudo wg show
interface: wg0
  public key: [REDACTED]
  private key: (hidden)
  listening port: 51820

peer: [REDACTED]
  allowed ips: 10.8.0.2/32
root@UbuntuServerVPN:/etc/wireguard#
```

Figure 8: sudo wg-quick up wg0

5.4 Router Configuration

Port Forwarding Configuration

- Accessed router administration interface
- Created port forwarding rule:
 - - External Port: 51820 (UDP)
 - - Internal IP: 192.168.1.97
 - - Internal Port: 51820 (UDP)

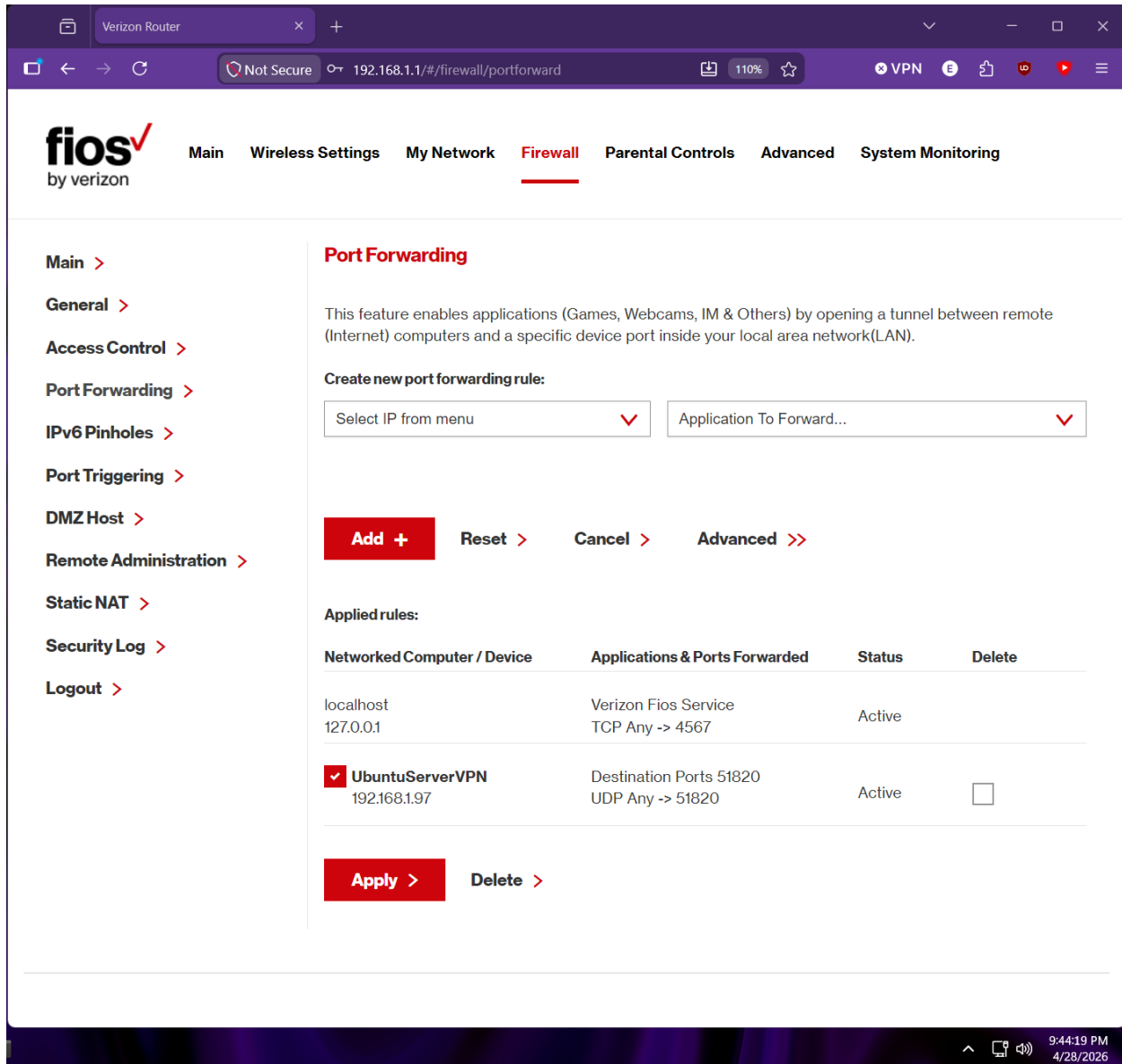


Figure 9: Router Port Forwarding Configuration

5.5 Client Setup

Windows Client Configuration

- Downloaded and installed WireGuard for Windows
- Created client configuration file with:
 - - Client private key
 - - Client VPN IP: 10.8.0.2/24
 - - Server public key
 - - Endpoint: X.X.X.X:51820
 - - AllowedIPs: 0.0.0.0/0 (route all traffic through VPN)
 - - PersistentKeepalive: 25

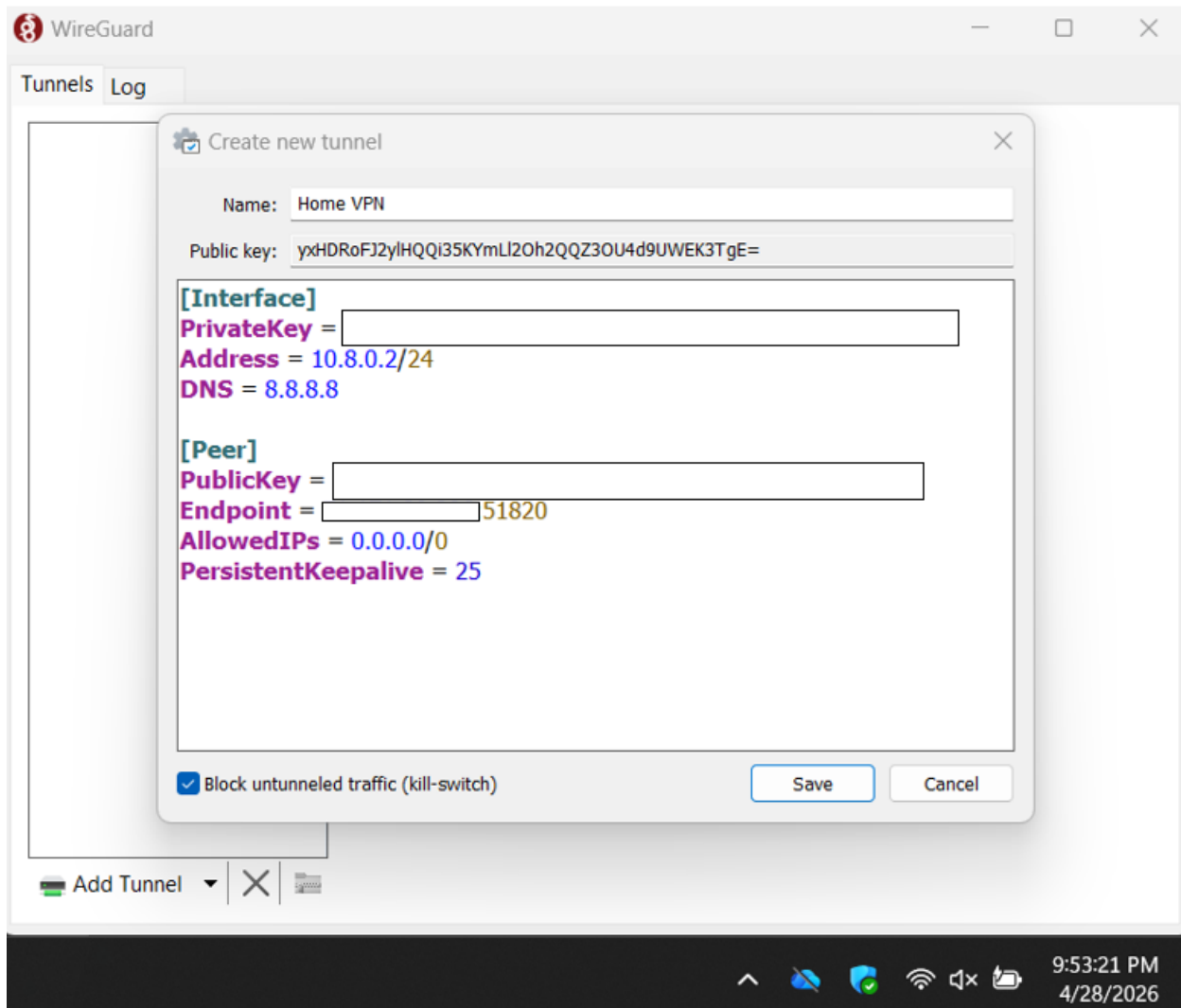


Figure 10: Client VPN Configuration

6. Firewall Rules and Security Configuration

6.1 Server-Side Firewall Rules

The VPN server implements multiple layers of firewall protection using Linux iptables. These rules control traffic flow, enable network address translation, and provide defense against unauthorized access.

NAT/MASQUERADE Rules:

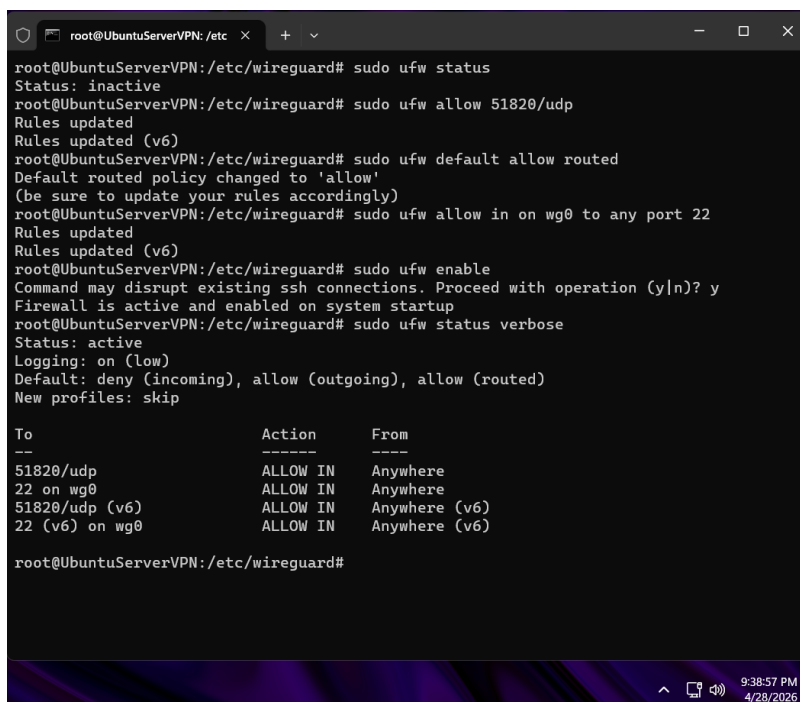
- Rule: `iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE`
- Purpose: Translates VPN client addresses (10.8.0.0/24) to the server's LAN IP for internet access
- Security Implication: Hides internal VPN addressing from external networks

FORWARD Chain Rules:

- Rule: `iptables -A FORWARD -i wg0 -j ACCEPT`
- Purpose: Allows forwarding of traffic from VPN interface

INPUT Chain Rules:

- Implicit rule: Accept UDP port 51820 (WireGuard listen port)
- Default policy: DROP for unauthorized traffic



```

root@UbuntuServerVPN:/etc/wireguard# sudo ufw status
Status: inactive
root@UbuntuServerVPN:/etc/wireguard# sudo ufw allow 51820/udp
Rules updated
Rules updated (v6)
root@UbuntuServerVPN:/etc/wireguard# sudo ufw default allow routed
Default routed policy changed to 'allow'
(be sure to update your rules accordingly)
root@UbuntuServerVPN:/etc/wireguard# sudo ufw allow in on wg0 to any port 22
Rules updated
Rules updated (v6)
root@UbuntuServerVPN:/etc/wireguard# sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
root@UbuntuServerVPN:/etc/wireguard# sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), allow (routed)
New profiles: skip

To Action From
--
51820/udp ALLOW IN Anywhere
22 on wg0 ALLOW IN Anywhere
51820/udp (v6) ALLOW IN Anywhere (v6)
22 (v6) on wg0 ALLOW IN Anywhere (v6)

root@UbuntuServerVPN:/etc/wireguard#

```

Figure 11: Enabling and Configuring Firewall Rules

7. Testing and Verification

7.1 Local Network Testing

Initial testing was conducted from within the home network to verify basic functionality before external testing.

- Connected laptop to home WiFi
- Activated VPN connection in WireGuard client
- Verified successful handshake with server
- Confirmed IP address assignment (10.8.0.2)
- Tested internet connectivity through tunnel

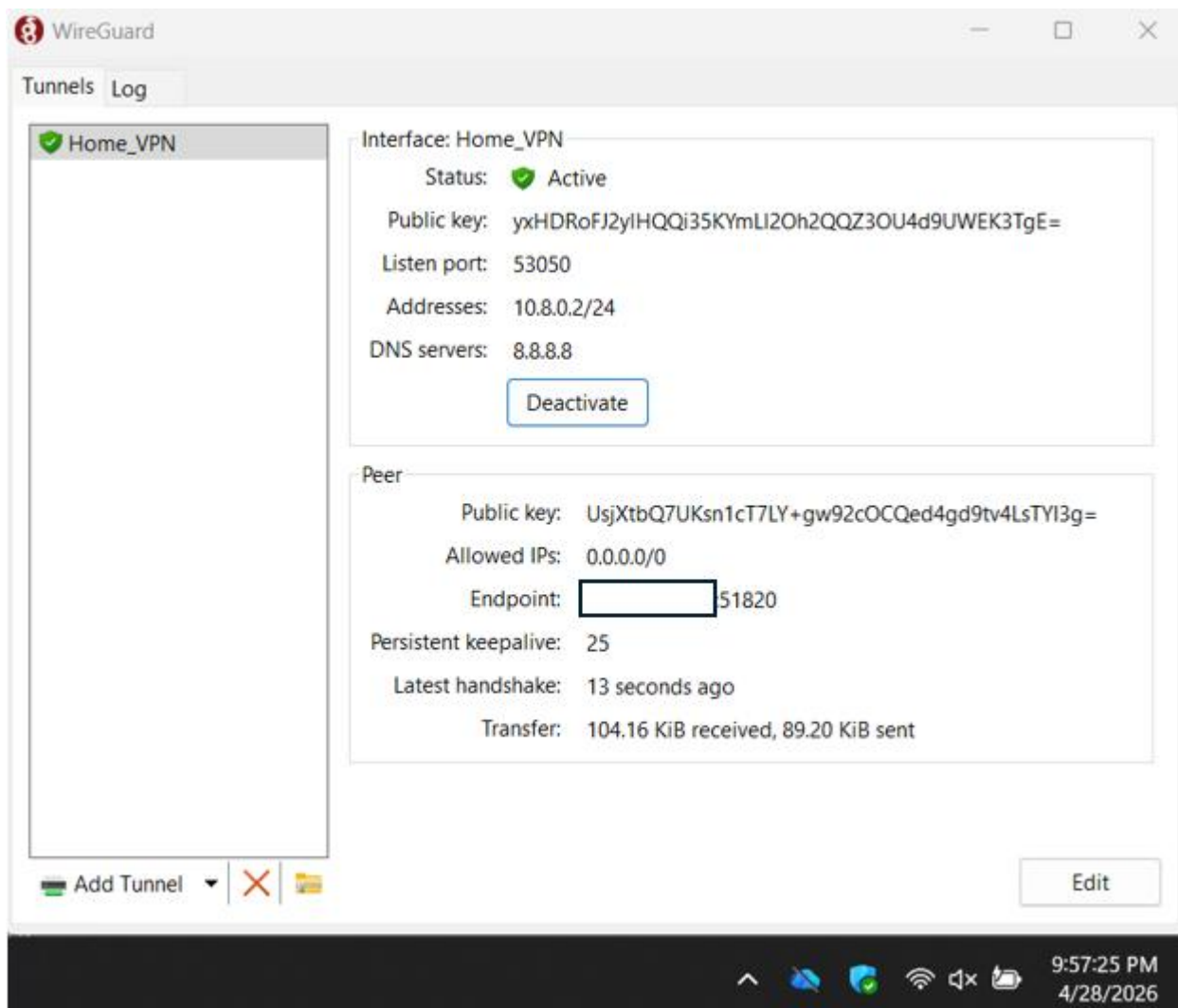


Figure 12: WireGuard client showing active status

7.2 Remote Network Testing

External testing verified VPN functionality from outside the home network, simulating real-world remote access scenarios.

- Disconnected laptop from home network
- Connected to external network (mobile hotspot / campus WiFi)
- Verified different public IP address (not home IP)
- Activated VPN connection
- Confirmed successful connection through router port forwarding
- Verified IP address changed to home public IP

Before VPN Connection:

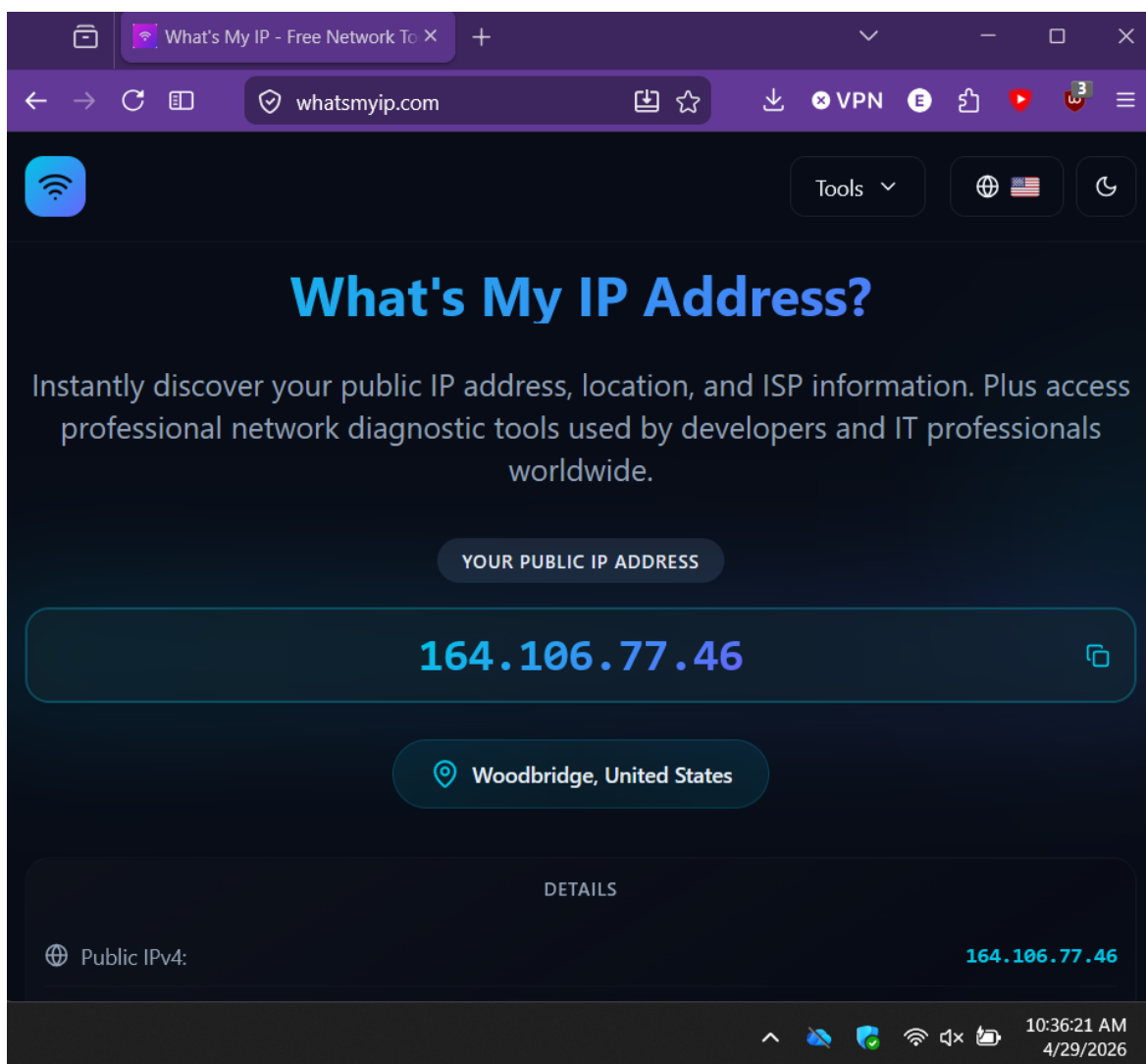


Figure 13: Client IP before connection

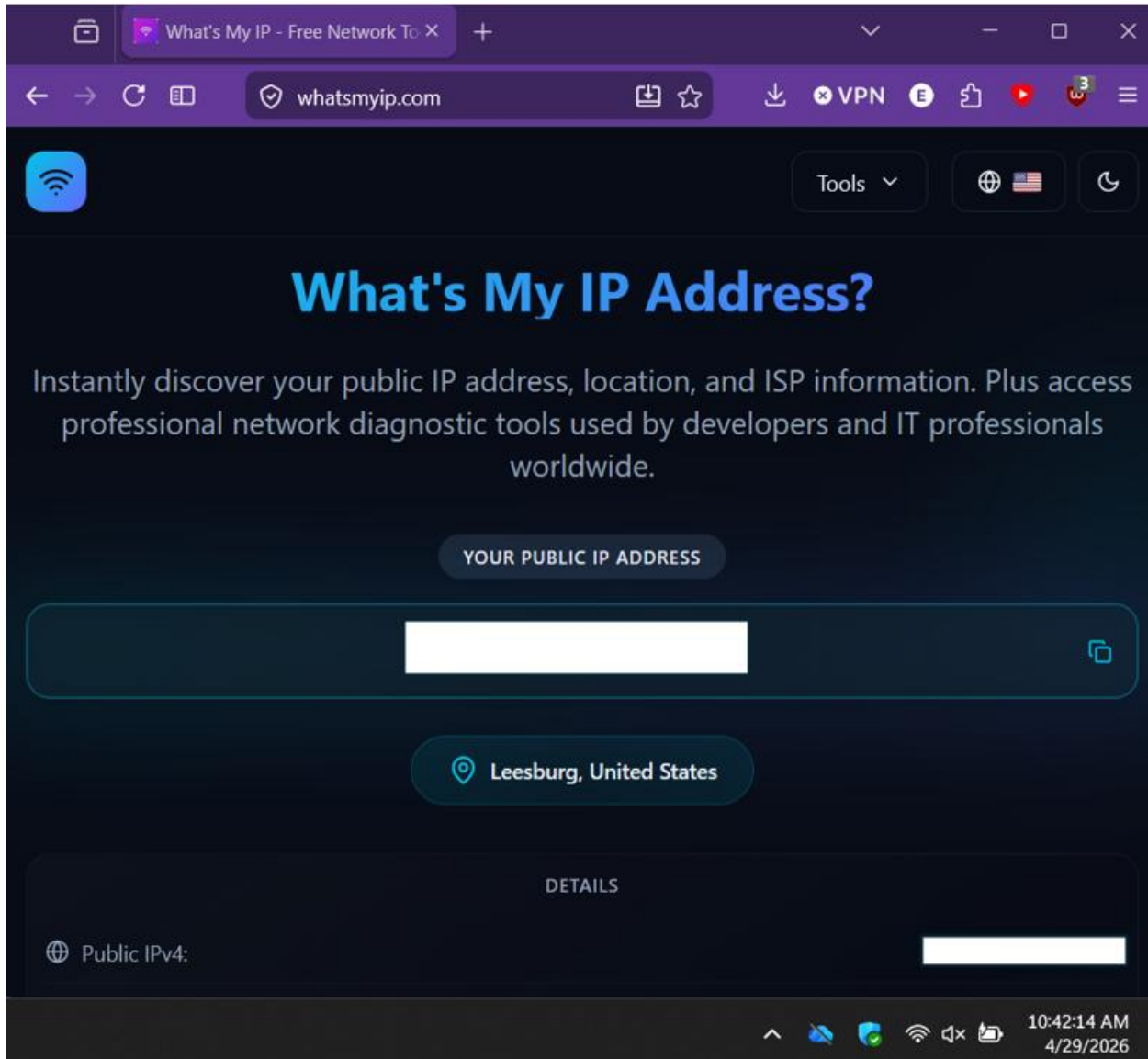
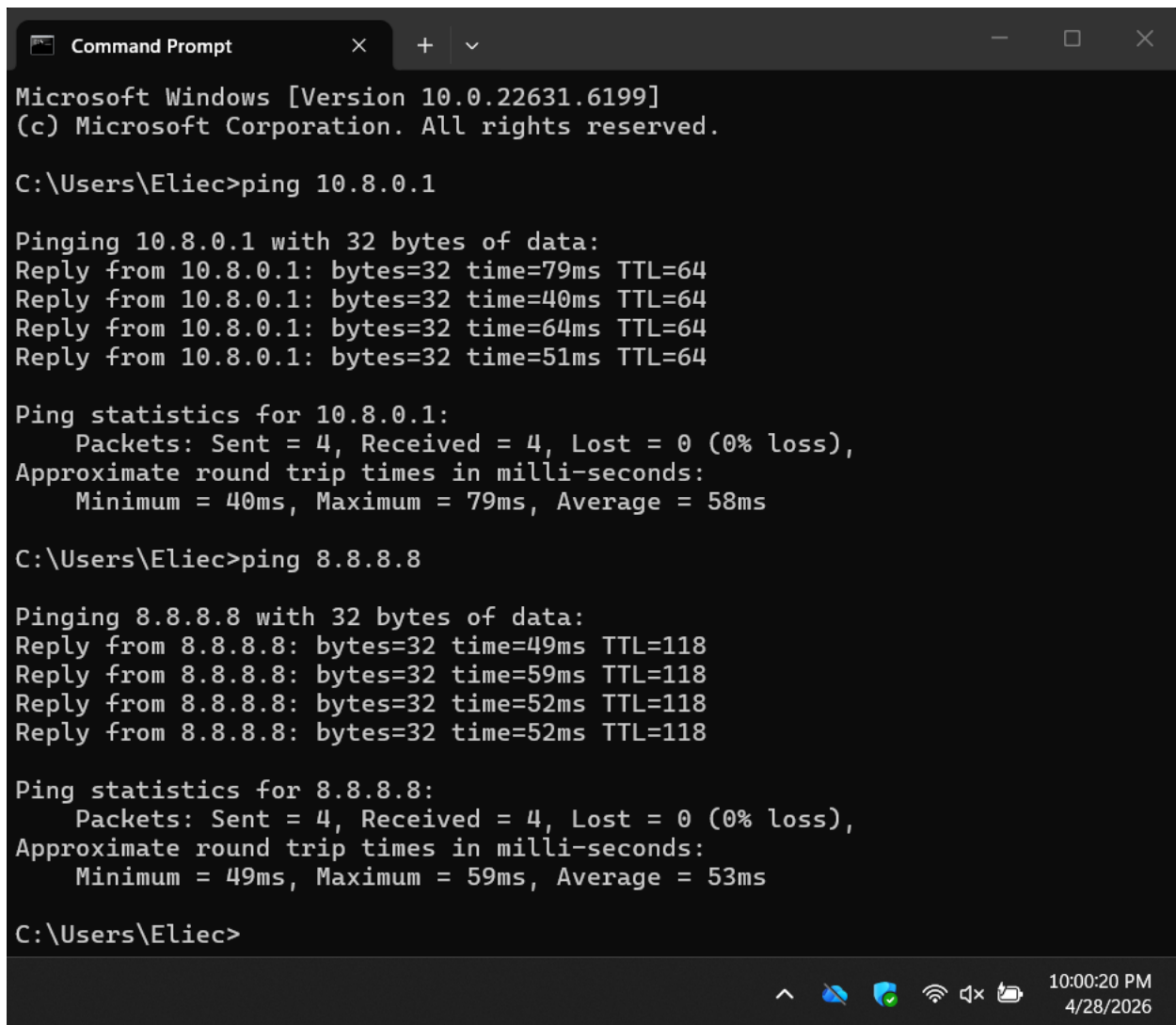
After VPN Connection:

Figure 14: Client IP after connection

7.3 Connectivity Verification

Command-line testing validated network functionality:

- ping 10.8.0.1 - Verified VPN server reachability
- ping 8.8.8.8 - Confirmed internet connectivity
- nslookup google.com - Tested DNS resolution
- tracert google.com - Verified traffic routing through home network



```
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Eliec>ping 10.8.0.1

Pinging 10.8.0.1 with 32 bytes of data:
Reply from 10.8.0.1: bytes=32 time=79ms TTL=64
Reply from 10.8.0.1: bytes=32 time=40ms TTL=64
Reply from 10.8.0.1: bytes=32 time=64ms TTL=64
Reply from 10.8.0.1: bytes=32 time=51ms TTL=64

Ping statistics for 10.8.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 40ms, Maximum = 79ms, Average = 58ms

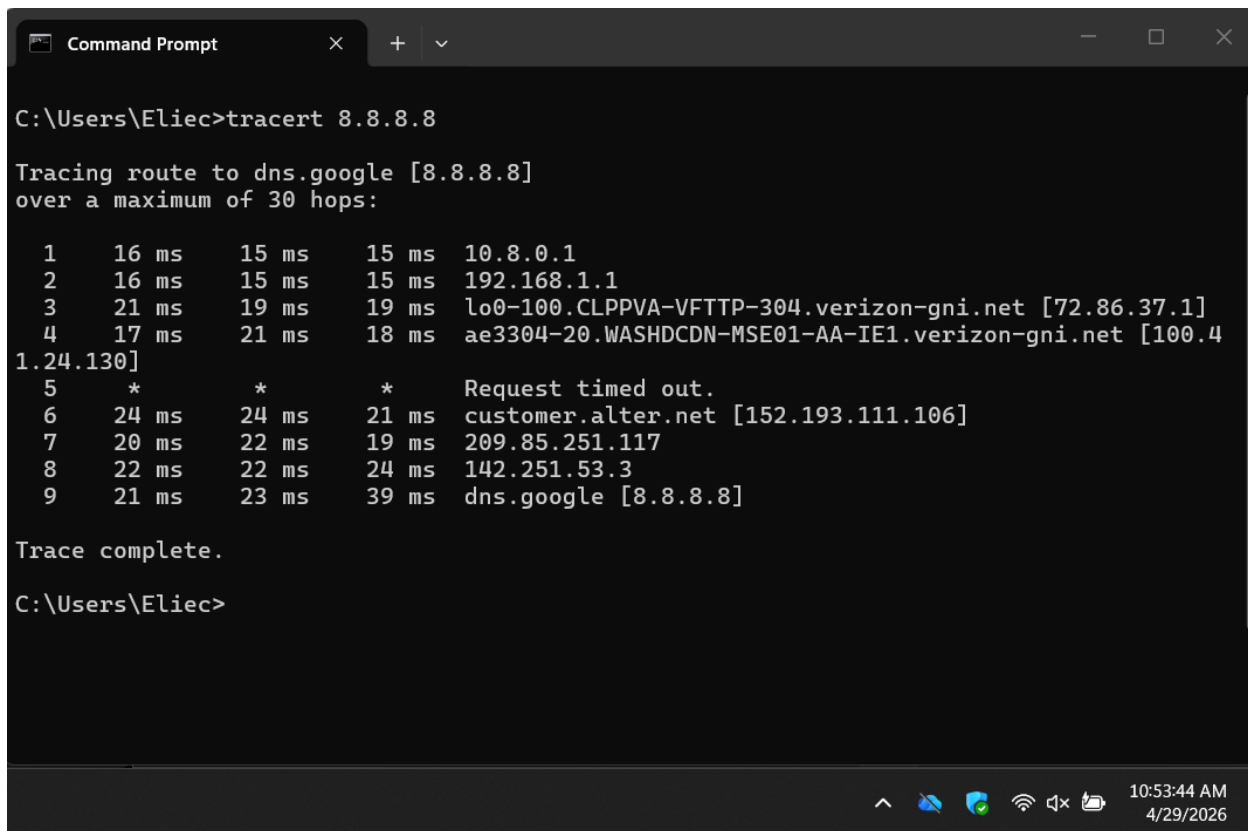
C:\Users\Eliec>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=49ms TTL=118
Reply from 8.8.8.8: bytes=32 time=59ms TTL=118
Reply from 8.8.8.8: bytes=32 time=52ms TTL=118
Reply from 8.8.8.8: bytes=32 time=52ms TTL=118

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 49ms, Maximum = 59ms, Average = 53ms

C:\Users\Eliec>
```

Figure 15: Results of pings while connected to VPN



```
C:\Users\Eliec>tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

  1    16 ms    15 ms    15 ms  10.8.0.1
  2    16 ms    15 ms    15 ms  192.168.1.1
  3    21 ms    19 ms    19 ms  lo0-100.CLPPVA-VFTTP-304.verizon-gni.net [72.86.37.1]
  4    17 ms    21 ms    18 ms  ae3304-20.WASHDCDN-MSE01-AA-IE1.verizon-gni.net [100.4
1.24.130]
  5     *         *         *      Request timed out.
  6    24 ms    24 ms    21 ms  customer.alter.net [152.193.111.106]
  7    20 ms    22 ms    19 ms  209.85.251.117
  8    22 ms    22 ms    24 ms  142.251.53.3
  9    21 ms    23 ms    39 ms  dns.google [8.8.8.8]

Trace complete.

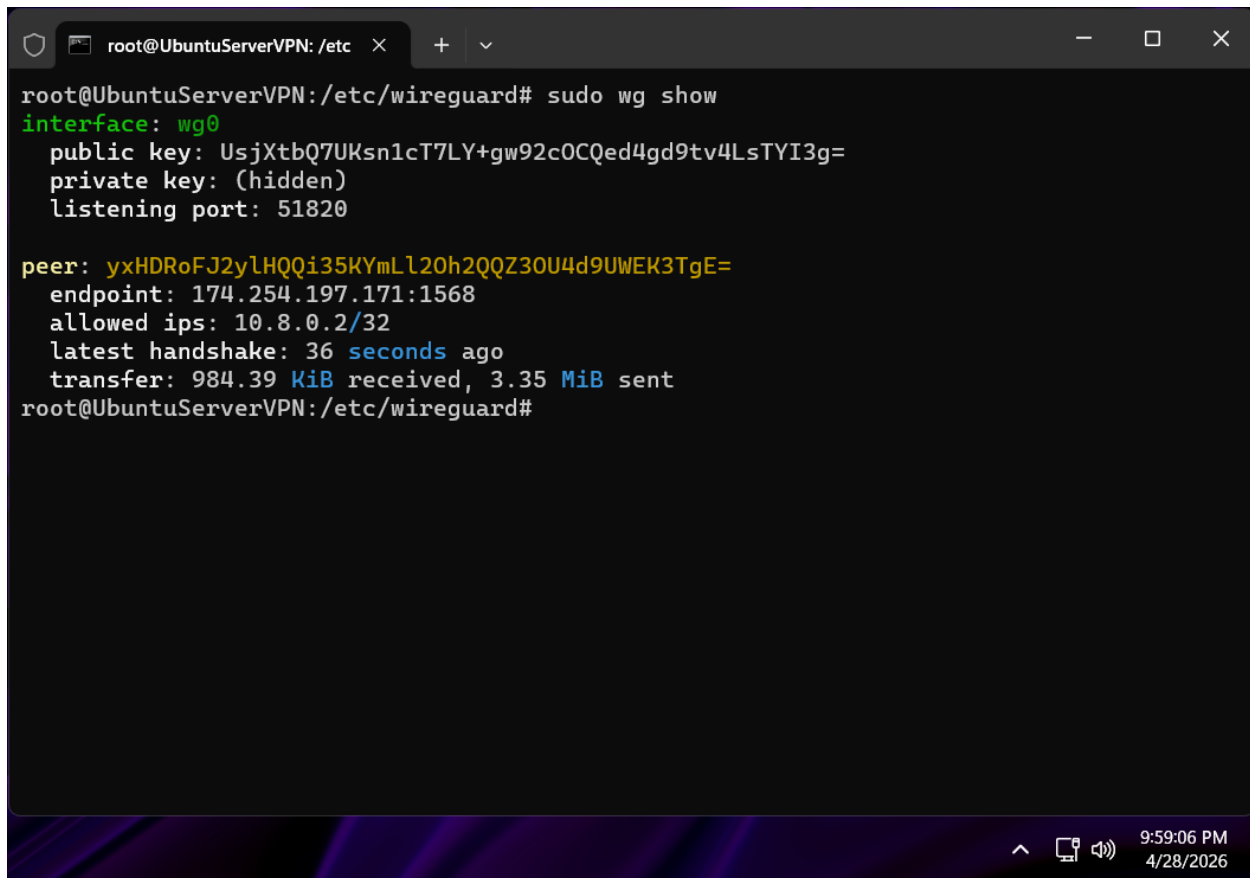
C:\Users\Eliec>
```

Figure 16: Results of tracert after connection to VPN

7.4 Server-Side Verification

Server logs and status confirmed client connections:

- `sudo wg show` - Displayed active peer with recent handshake
- Verified data transfer counters (bytes sent/received)
- Confirmed endpoint address matches client's public IP

A terminal window titled 'root@UbuntuServerVPN: /etc' showing the output of the 'sudo wg show' command. The output displays the configuration for the 'wg0' interface, including its public key, listening port (51820), and details for an active peer. The peer's endpoint is 174.254.197.171:1568, and it has been active for 36 seconds, having transferred 984.39 KiB received and 3.35 MiB sent. The terminal background is dark, and the text is in various colors (green for interface, yellow for peer, blue for time and data). The window has standard Ubuntu window controls at the top and a system tray at the bottom right showing the time as 9:59:06 PM on 4/28/2026.

```
root@UbuntuServerVPN:/etc/wireguard# sudo wg show
interface: wg0
  public key: UsjXtbQ7UKsn1cT7LY+gw92c0CQed4gd9tv4LsTYI3g=
  private key: (hidden)
  listening port: 51820

peer: yxHDRoFJ2ylHQQi35KYmLl20h2QQZ30U4d9UWEK3TgE=
  endpoint: 174.254.197.171:1568
  allowed ips: 10.8.0.2/32
  latest handshake: 36 seconds ago
  transfer: 984.39 KiB received, 3.35 MiB sent
root@UbuntuServerVPN:/etc/wireguard#
```

Figure 17: sudo wg show has active peer

8. Conclusion

This VPN solution has strong cryptographic protection using ChaCha20Poly1305 encryption and Curve25519 key exchange, demonstrating modern security best practices. Multi-layer firewall protection through both router-level and server level controls adds a defense-in-depth strategy that strengthens the security of the home network.

This project successfully deployed a functional VPN server for secure remote access to a home network. The implementation uses WireGuard VPN technology on an Ubuntu Linux virtual machine, with firewall configuration, network address translation, and encrypted tunneling.